SESION N° 4

Tema: Diagramas UML, Constructores, la Referencia this y Métodos Especializados

I. Objetivo de la Guía

Modelar las entidades principales de un sistema mediante diagramas de clases UML y codificarlas correctamente en Java implementando constructores, la referencia this, y métodos especializados de acceso y modificación (accessors y mutators).

1. Diagramas de Clases UML

Herramienta de modelado visual que permite representar la estructura de un sistema orientado a objetos, mostrando las clases, sus atributos (con su visibilidad: + público o − privado), métodos y las relaciones entre ellas.

2. Constructores y la Referencia this

- Constructor: Método especial que se ejecuta automáticamente al crear un objeto para inicializar sus atributos.
- this: Palabra reservada que hace referencia al objeto actual de la clase. Muy útil para diferenciar los atributos de los parámetros cuando tienen el mismo nombre.
- this es como decir "MI" en vez de repetir tu propio nombre: dentro de un método o constructor, this apunta automáticamente "a mí mismo" (el objeto sobre el que se está ejecutando ese código).

3. Métodos Especializados

- Accessors (getters): Permiten consultar u obtener el valor de un atributo privado.
- Mutators (setters): Permiten modificar o asignar un nuevo valor a un atributo privado.
- toString(): Método para representar el estado del objeto en formato de texto.

Errores Comunes al Empezar

ERROR TIPICO	COMO EVITARLO
Nombrar el parámetro del constructor igual que el atributo, pero olvidar escribir this.	Cuando el parámetro se llama igual que el atributo, debes usar this.atributo = atributo; si no, Java asume que hablas del parámetro en ambos lados y el atributo se queda con su valor por defecto.
Declarar los atributos como public en vez de private.	Los atributos deben ser private; el acceso controlado se hace solo a través de métodos getters y setters — eso es el encapsulamiento.
Escribir un getter sin la palabra return.	Todo getter necesita return seguido del atributo que quiere devolver, o el método no entrega ningún valor.

II. Actividades Prácticas Paso a Paso

Codificación Orientada a Objetos

Lleva el modelo anterior al código fuente en Java. Implementa la clase Estudiante.java aplicando encapsulamiento, constructores, la referencia this, y los métodos getters, setters y toString.
Código completo — Estudiante.java:

Código: Estudiante.java

```java
public class Estudiante {
    // Atributos privados (Encapsulamiento)
    private int id;
    private String nombre;
    private int puntos;

    // Constructor por defecto
    public Estudiante() {
    }

    // Constructor parametrizado utilizando la referencia 'this' (Trabajo 2)
    public Estudiante(int id, String nombre, int puntos) {
        this.id = id;
        this.nombre = nombre;
        this.puntos = puntos;
    }

    // Métodos Accessors (Getters) y Mutators (Setters) (Trabajo 2)
    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public int getPuntos() {
        return puntos;
    }

    public void setPuntos(int puntos) {
        this.puntos = puntos;
    }

    // Método especializado toString para mostrar la información del objeto
    @Override
    public String toString() {
        return "Estudiante [ID=" + id + ", Nombre=" + nombre + ", Puntos=" + puntos + "]";
    }
}
```

EXPLICACIÓN DEL CÓDIGO 1

public class Estudiante {
Declara la clase Estudiante.
// Atributos privados (Encapsulamiento)
Comentario que indica que los atributos que siguen están encapsulados (son privados).
private int id;
Declara el atributo privado id, de tipo entero.
private String nombre;
Declara el atributo privado nombre, de tipo texto (String).
private int puntos;
Declara el atributo privado puntos, de tipo entero.
// Constructor por defecto
Comentario que anuncia el constructor por defecto (sin parámetros).
public Estudiante() {
Constructor por defecto: permite crear un Estudiante sin indicar datos iniciales (new Estudiante()).
}
Cierra el constructor por defecto (no hace nada dentro; los atributos quedan con sus valores por defecto: 0, null, 0).
// Constructor parametrizado utilizando 'this'
Comentario que anuncia el constructor parametrizado y menciona el uso de this.
public Estudiante(int id, String nombre, int puntos) {
Constructor parametrizado: recibe id, nombre y puntos para inicializar un nuevo Estudiante de una sola vez.
this.id = id;
Usa this.id para referirse al atributo del objeto, y lo iguala al parámetro id (que tiene el mismo nombre).
this.nombre = nombre;
Usa this.nombre para referirse al atributo del objeto, y lo iguala al parámetro nombre.
this.puntos = puntos;
Usa this.puntos para referirse al atributo del objeto, y lo iguala al parámetro puntos.
}
Cierra el constructor parametrizado.
// Métodos Accessors (Getters) y Mutators (Setters)
Comentario que anuncia el bloque de métodos accessors (getters) y mutators (setters).
public int getId() {
Getter de id: método público que permite CONSULTAR el valor del atributo privado id desde fuera de la clase.
return id;
Devuelve el valor actual de id.
}
Cierra el método getId().
public void setId(int id) {
Setter de id: método público que permite MODIFICAR el valor del atributo privado id desde fuera de la clase.
this.id = id;
Asigna el nuevo valor recibido al atributo this.id.
}
Cierra el método setId().
public String getNombre() {
Getter de nombre.
return nombre;
Devuelve el valor actual de nombre.
}
Cierra el método getNombre().
public void setNombre(String nombre) {
Setter de nombre.
this.nombre = nombre;

Asigna el nuevo valor recibido al atributo this.nombre.
}
Cierra el método setNombre().
public int getPuntos() {
Getter de puntos.
return puntos;
Devuelve el valor actual de puntos.
}
Cierra el método getPuntos().
public void setPuntos(int puntos) {
Setter de puntos.
this.puntos = puntos;
Asigna el nuevo valor recibido al atributo this.puntos.
}
Cierra el método setPuntos().
// Método especializado toString...
Comentario que anuncia el método especializado toString().
@Override
Anotación @Override: indica que este método reemplaza al toString() heredado de la clase Object.
public String toString() {
Declara el método toString(), que debe devolver un String con la información del objeto.
return "Estudiante [ID=" + id + ...];
Construye y devuelve un texto que combina el id, el nombre y los puntos actuales del objeto.
}
Cierra el método toString().
}
Cierra la clase Estudiante.

Antes de seguir:

Si en el constructor de Estudiante escribieras puntos = puntos; en vez de this.puntos = puntos;, ¿qué valor tendría el atributo puntos del objeto recién creado?

Respuesta: 0 (el valor por defecto de un int) — sin this., Java entiende que le estás asignando el parámetro a sí mismo, y el atributo de la clase nunca recibe el valor real que le pasaste al crear el objeto.

Clase de ejecución — PrincipalEstudiante.java

Crea una clase de ejecución para probar la correcta creación de objetos utilizando el constructor:

Código: PrincipalEstudiante.java

```
public class PrincipalEstudiante {
    public static void main(String[] args) {
        // Creación de objeto usando el constructor parametrizado
        Estudiante alumno1 = new Estudiante(1, "Carlos Mendoza", 450);

        // Uso de métodos mutators para actualizar datos
        alumno1.setPuntos(500);

        // Mostrar datos usando el método toString y accessors
        System.out.println(alumno1.toString());
        System.out.println("Nombre consultado con Getter: " + alumno1.getNombre());
```

```
    }
}
```

EXPLICACIÓN DEL CÓDIGO 2

public class PrincipalEstudiante {
Declara la clase PrincipalEstudiante, donde se probará la clase Estudiante.
public static void main(String[] args) {
Punto de entrada del programa: aquí comienza la ejecución cuando se corre esta clase.
// Creación de objeto usando el constructor parametrizado
Comentario que anuncia la creación de un objeto usando el constructor parametrizado.
Estudiante alumno1 = new Estudiante(1, "Carlos Mendoza", 450);
Crea un nuevo objeto Estudiante usando el constructor parametrizado, y lo guarda en la variable local alumno1.
//Uso de métodos mutators para actualizar datos
Comentario que anuncia el uso de un método mutator.
alumno1.setPuntos(500);
Llama al setter setPuntos(500): modifica el atributo puntos del objeto alumno1, que antes valía 450.
// Mostrar datos usando el método toString y accessors
Comentario que anuncia el uso de toString() y de un accessor.
System.out.println(alumno1.toString());
Imprime en consola el resultado de alumno1.toString(), que muestra id, nombre y el puntaje YA actualizado (500).
System.out.println("Nombre consultado con Getter: " + alumno1.getNombre());
Imprime en consola un texto fijo seguido del resultado de alumno1.getNombre(): demuestra el uso de un accessor (getter).
}
Cierra el método main.
}
Cierra la clase PrincipalEstudiante.

Salida esperada en consola:

```
Estudiante [ID=1, Nombre=Carlos Mendoza, Puntos=500]
Nombre consultado con Getter: Carlos Mendoza
PS E:\Semestre IV\Taller de lenguajes de programacion\sesion 4>
```

Resumen de Conceptos Clave

- **Diagrama UML**: Representación visual de clases, atributos (+ público / − privado) y métodos.
- **Constructor**: Se ejecuta automáticamente al crear un objeto con new; inicializa los atributos.
- **this**: Referencia al objeto actual; diferencia atributos de parámetros con el mismo nombre.
- **Getters (accessors)**: Métodos que permiten LEER un atributo privado desde fuera de la clase.
- **Setters (mutators)**: Métodos que permiten MODIFICAR un atributo privado desde fuera de la clase.
- **Encapsulamiento**: Atributos private + acceso controlado mediante getters y setters.
- **toString()**: Método que convierte el objeto en un texto legible para mostrar su estado.